BSG Institute

Programa Certified AI-LLM Solution Architect

Curso Diseño de Infraestructura Escalable

Claves para construir plataformas eficientes y adaptables en la nube

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 25 años de experiencia
- PGP Artificial Intelligence & Machine Leaning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialista en MLOps de la Universidad de Duke
- Estudiante de Deep Learning for Healthcare Specialization
- Director de Inteligencia Artificial e Innovacion Universidad EAN/Colombia

arojaspa@outlook.com
@arojaspa
https://www.linkedin.com/in/arojaspa/
+573005906373



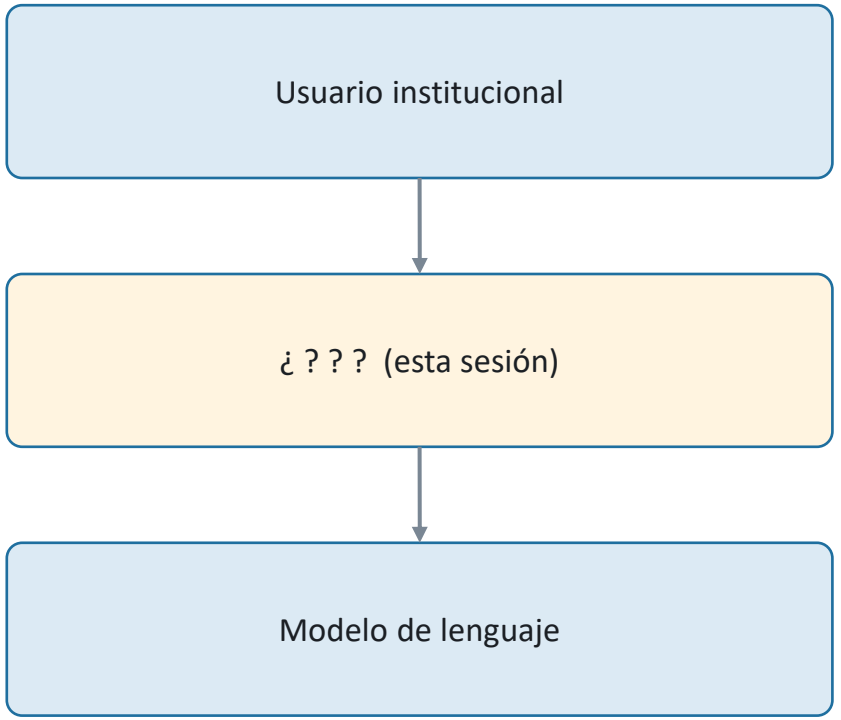
Arquitecturas Base para LLMs en la Nube

Capítulo 1 · Fundamentos de Infraestructura para LLMs · Sesión 1

Diseño de Infraestructura Escalable para LLMs

¿Qué hay entre el usuario y el modelo?

- Caso transversal: asistente institucional de conocimiento para una universidad
  - 5.000 usuarios potenciales · 100 concurrentes (hipótesis de diseño)
  - Disponibilidad objetivo 99,9 % · acceso autenticado · auditoría
  - Documentos institucionales · protección de datos · presupuesto limitado
- Los valores son supuestos académicos, no mediciones de carga
- Pregunta guía de la sesión: ¿qué componentes deben existir, dónde viven y por qué?



Resultados de aprendizaje y agenda

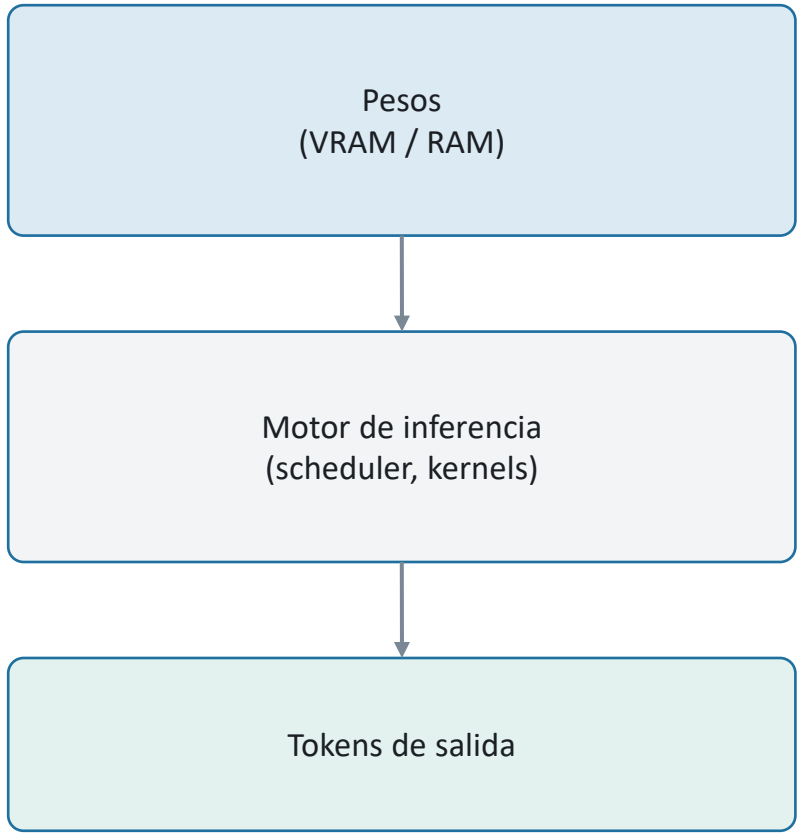
RA	Al finalizar podrá…	Bloque
RA1	Explicar el ciclo de vida de una solicitud: prefill, decode, KV cache	Unidad A · Demo 1
RA2	Identificar componentes y plano de control vs. plano de datos	Unidad B
RA3	Comparar inferencia gestionada y autogestionada con criterios	B · C · Demo 2
RA4	Estimar memoria: pesos + KV cache según precisión y concurrencia	Unidad C
RA5	Interpretar TTFT, TPOT, throughput y saturación	F · Demo 4
RA6	Mapear categorías de servicio en AWS, Azure y Google Cloud	Unidad D · Demo 3
RA7	Asociar riesgos OWASP LLM 2025 con controles por capa	Unidad E
RA8	Diseñar y justificar una topología base para el caso	Taller

Unidad A · Fundamentos de inferencia

¿Qué recurso se agota primero cuando un LLM atiende a muchos usuarios?

Un LLM visto desde la infraestructura

- Dos piezas: archivo de pesos (miles de millones de parámetros) + motor de inferencia
- Cada token generado ≈ una pasada completa por la red: ~2 × N operaciones para N parámetros
- Cada paso de decode lee prácticamente todos los pesos desde memoria
- Consecuencia: el tamaño del modelo fija la memoria necesaria y la velocidad por token
- Servir un modelo = mantener pesos en memoria rápida + ejecutar el motor por cada token

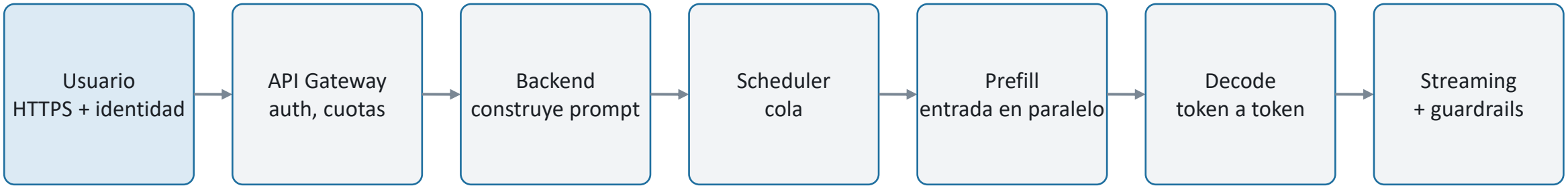


Tokens y context window

El token es…	Implicación de infraestructura
Unidad de facturación	Las APIs gestionadas cobran por token de entrada y de salida
Unidad de latencia	Cada token de salida es un paso de decode
Unidad de capacidad	Cada token activo ocupa Key-Value cache mientras la conversación vive

- Tokenizadores entrenados mayormente en inglés producen más tokens por palabra en español: medir con el tokenizador real
- Context window = máximo de tokens de entrada + salida por solicitud
- Una ventana grande no es gratis: cuesta dinero, alarga el prefill y ocupa memoria

Ciclo de vida de una solicitud de inferencia



- Pasos 1–3 y posprocesamiento: infraestructura clásica (identidad, red, aplicación)
- Cola, prefill y decode: específicos de LLM → núcleo de las unidades C y F
- Cada flecha es un punto de latencia y un punto de control de seguridad

Prefill y decode: dos fases, dos cuellos de botella

Fase	Qué hace	Recurso dominante	Métrica	Crece con
Prefill	Procesa todos los tokens de entrada en paralelo y llena la KV cache	Cómputo	TTFT	Longitud del prompt
Decode	Genera un token por paso, secuencialmente	Ancho de banda de memoria	TPOT	Longitud de la respuesta

RAG con documentos largos degrada el TTFT; respuestas largas degradan la latencia total por decode.

Key-Value cache: la memoria que limita la concurrencia

KV bytes/token = 2 × capas × kv_heads × d_head × bytes de precisión

- Guarda keys y values de todos los tokens previos para no recalcularlos
- Crece linealmente con los tokens de contexto y con las secuencias simultáneas
- Por eso un servidor LLM suele quedarse sin memoria antes que sin cómputo
- Mitigaciones: grouped-query attention (menos kv_heads), PagedAttention, cuantización de la KV cache

Batching y concurrencia

Estrategia	Funcionamiento	Ventaja	Limitación
Sin batching	Una solicitud a la vez	Latencia individual mínima	GPU subutilizada, costo por token alto
Batching estático	Agrupa N y espera a que terminen todas	Simple	Las respuestas cortas esperan a las largas
Batching continuo	Entran y salen secuencias en cada paso de decode	Alto throughput, latencia razonable	Requiere motor especializado

Trade-off: más secuencias por lote → más throughput y menor costo por token, pero mayor TPOT por usuario.

Entrenamiento, fine-tuning e inferencia

Actividad	Propósito	Recursos	Frecuencia	En el caso
Preentrenamiento	Aprender el modelo base	Miles de aceleradores, semanas o meses	Una vez	No: se adquiere el modelo
Fine-tuning	Especializar comportamiento o formato	Uno o pocos aceleradores (LoRA)	Ocasional	Probablemente no al inicio
Inferencia	Responder solicitudes	Memoria y ancho de banda 24/7	Continua	Sí: directa o vía proveedor

Recomendación: para conocimiento institucional cambiante, RAG antes que fine-tuning.

Modelos propietarios, open-weight y open source

Tipo	Acceso a pesos	Ejecución	Implicaciones
Propietario	No	Solo vía API del proveedor	Mínima operación; dependencia y términos de datos del proveedor
Open-weight	Sí, con licencia que puede restringir usos	Self-hosting o servicio gestionado	Control y portabilidad; revisar la licencia para uso institucional
Open source estricto	Pesos, código e información de datos	Igual que open-weight	Menos común; Open Source AI Definition (OSI)

¿Comprar o producir inferencia?

	API gestionada (comprar)	Self-hosting (producir)
Qué se paga	Tokens de entrada y salida	GPU por hora + operación del equipo
Qué se opera	Prompt, cuotas, integración	GPU, motor de serving, escalado, parches
Control de datos	Según términos y región del proveedor	Total, dentro de la red propia
Tiempo a producción	Días	Semanas o meses

La decisión no es binaria: la Unidad B la ubica dentro de una arquitectura completa.

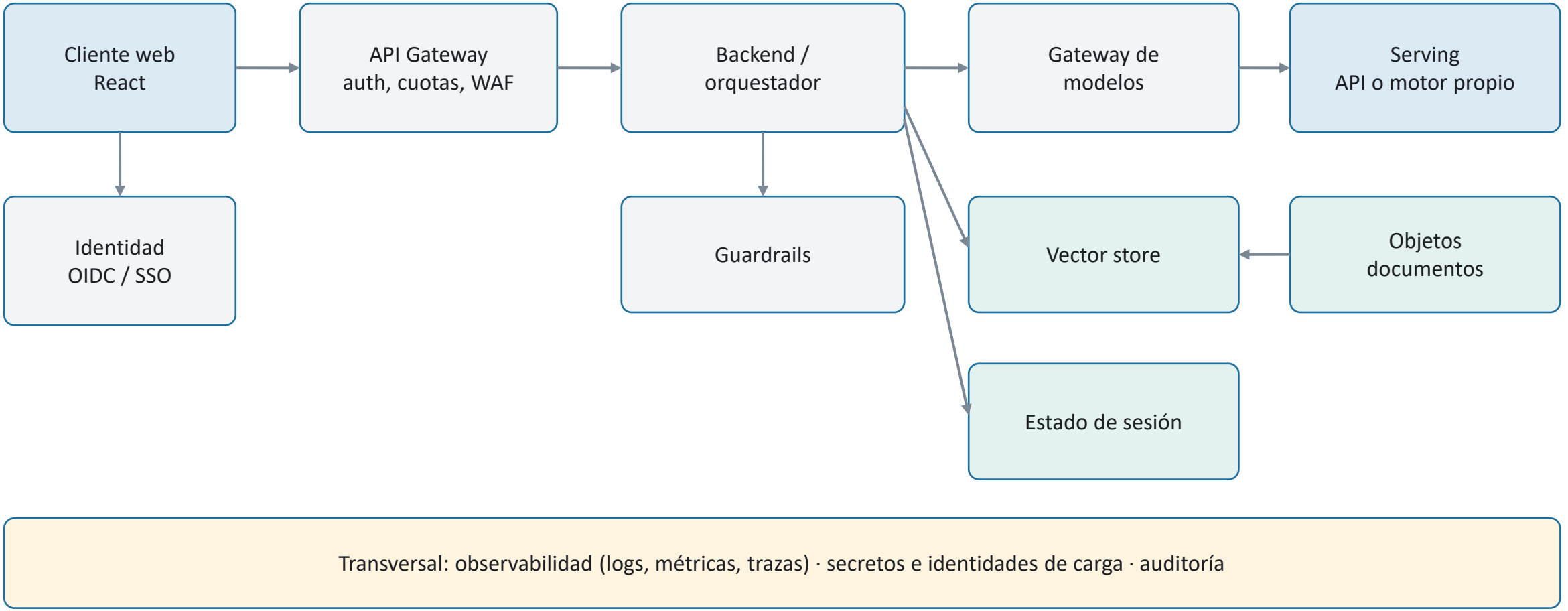
Demo 1 · Anatomía de una solicitud (Ollama local)

- Modelo open-weight pequeño (1–3 B, cuantizado) ejecutado en CPU
- Pregunta corta → observar TTFT, TPOT y tokens/s en la interfaz
- Misma pregunta con 3 párrafos de reglamento delante → el TTFT sube, el TPOT casi no cambia
- Observar la RAM usada y compararla con el tamaño del modelo
- Repositorio: labs/00-local-ollama (simulador disponible si Ollama falla)

Unidad B · Arquitecturas base

¿Qué debe existir entre el usuario y el modelo para que el asistente sea seguro, observable y escalable?

Arquitectura de referencia (D-06)



Responsabilidad de cada componente

Componente	Responsabilidad	Falla típica	Métrica clave
API Gateway	Autenticación, límites por usuario, perímetro	Rate limit mal dimensionado	Rechazos 429
Backend / orquestador	Construir el prompt, recuperar contexto, invocar herramientas	Prompts excesivos, timeouts encadenados	Latencia end-to-end
Guardrails	Filtrar entradas y salidas indebidas	Falsos positivos o negativos	Tasa de bloqueo
Gateway de modelos	Enrutar modelos o proveedores, reintentos, cuotas	Punto único de fallo	Errores por proveedor
Serving	Ejecutar la inferencia	Saturación de memoria, colas	TTFT, TPOT, cola
Vector store	Búsqueda semántica del contexto	Índice desactualizado o sin permisos	Relevancia y latencia
Observabilidad	Trazabilidad y diagnóstico	Registrar datos personales	Cobertura de trazas

Plano de control y plano de datos

Plano de control

Despliegue de modelos · versiones de prompts · políticas · cuotas · identidades · configuración

Plano de datos

Cada solicitud de usuario: gateway → backend → serving → vector store → respuesta

Separarlos permite escalar el plano de datos sin exponer la administración, con permisos distintos en cada uno.

Monolítico o distribuido

Criterio	Monolito	Distribuido
Complejidad inicial	Baja	Mayor: red, contratos, despliegues
Escalado	Todo escala junto	Cada componente según su cuello de botella
Aislamiento de fallos	Un fallo afecta a todo	Contenido con timeouts y circuit breakers
Adecuado para	Prototipo, laboratorio local	Producción con cargas variables

Recomendación: separar el serving del backend desde el inicio (perfiles GPU/memoria vs. CPU/E/S).

Espectro: gestionado ↔ autogestionado

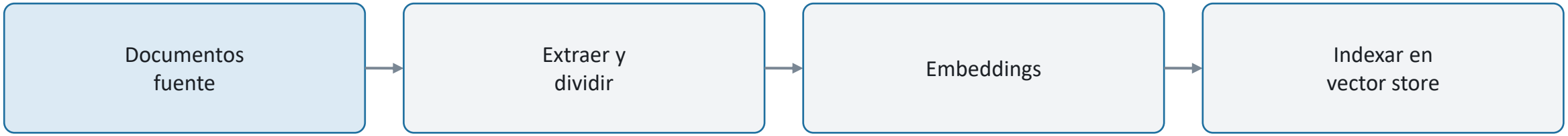
Modalidad	Quién opera la inferencia	Qué controla la organización	Cuándo conviene
VM con GPU	La organización	Todo	Investigación, aislamiento extremo
Kubernetes con GPU	La organización, sobre clúster gestionado	Motor, escalado, red	Varios modelos propios, equipo de plataforma
Contenedores serverless	Proveedor escala; organización aporta imagen	Imagen y configuración	Cargas variables, modelos pequeños
API de modelos (MaaS)	Proveedor	Prompt, parámetros, cuotas, región	Mayoría de casos al inicio
Runtime gestionado de agentes	Proveedor ejecuta agente, sesiones, memoria	Lógica, herramientas, políticas	Agentes con estado en producción

Patrones de integración

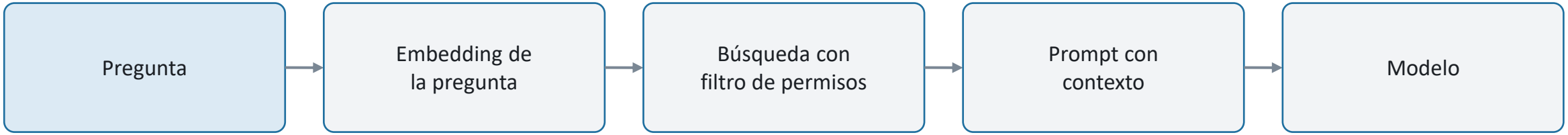
Patrón	Ventaja	Limitación	Uso en el caso
Síncrono	Simple	El usuario espera toda la generación	Llamadas internas cortas
Streaming (SSE / WebSocket)	Latencia percibida baja	Conexiones largas, timeouts de proxies	Conversación del asistente
Asíncrono con cola	Absorbe picos, reintentos	Mayor latencia, más componentes	Resúmenes de documentos largos
Event-driven	Desacoplamiento	Trazabilidad más difícil	Reindexar al publicar un reglamento

RAG como carga de trabajo

Ingesta (por lotes, por eventos)



Consulta (en línea, añade latencia y tokens)



Nuevos riesgos: documentos con permisos distintos e inyección indirecta a través del contenido recuperado.

Agentes como carga de trabajo

Cambio	Por qué importa a la infraestructura
Duración	Una tarea puede tomar muchos pasos y minutos: los timeouts de chat no sirven
Estado	Memoria de sesión y, a veces, de largo plazo
Identidad	Actúa sobre sistemas reales: identidad propia con mínimo privilegio
Costo variable	Número de llamadas al modelo por solicitud no fijo

Por eso los tres proveedores ofrecen runtimes gestionados específicos para agentes (Unidad D).

Puntos únicos de fallo y límites de escalabilidad

- Candidatos a punto único de fallo en D-06:
  - Gateway de modelos sin réplicas
  - Un solo proveedor o región de modelo
  - Vector store sin réplica multizona
  - Proveedor de identidad
- Los límites más frecuentes son contractuales: cuotas TPM (tokens/min) y RPM del proveedor
- Recomendación: documentar las cuotas como requisito no funcional, con alertas antes de alcanzarlas

Demo 2 · Misma API, dos backends

- Backend FastAPI con contrato común: /v1/chat/stream (SSE), /readyz
- Paso 1: proveedor = API nativa de Ollama
- Paso 2: cambiar solo variables de entorno a un endpoint compatible con OpenAI (/v1)
- El frontend, el contrato y las pruebas no cambian; cambia el adaptador
- Lección: aislar el proveedor reduce la dependencia y facilita la comparación multicloud

Unidad C · Infraestructura

Si la universidad aloja su propio modelo, ¿cuánta GPU necesita y dónde se ubica en la red?

CPU, GPU y jerarquía de memoria

Recurso	Fortaleza	Papel en inferencia LLM
CPU	Pocos núcleos potentes	Backend, tokenización, RAG; inferencia solo de modelos pequeños cuantizados
GPU	Miles de unidades paralelas + VRAM de alto ancho de banda	Prefill y decode de modelos medianos y grandes
RAM	Capacidad alta, menor ancho de banda	Inferencia en CPU, carga de pesos, offloading lento
Disco / objetos	Capacidad muy alta, lento	Guardar pesos; determina el arranque en frío

Límite superior con batch 1: tokens/s ≈ ancho de banda de memoria / tamaño de los pesos

Método de dimensionamiento de memoria

$$M \approx N \times b \; + \; S \times L \times KV(\text{bytes/token}) \; + \; M_{\text{runtime}}$$

Símbolo	Significado
$N \times b$	Pesos: parámetros × bytes por parámetro (FP16 = 2, INT8 = 1, INT4 ≈ 0,5 + escalas)
S	Secuencias activas simultáneas
L	Tokens de contexto por secuencia
$KV(\text{bytes/token})$	2 × capas × kv_heads × d_head × bytes
M_{runtime}	Activaciones, buffers y fragmentación: margen del 10–20 %

Ejemplo resuelto: el asistente universitario

- Supuestos: modelo de 8 B tipo Llama 3.1 8B (32 capas, 8 kv_heads, d_head 128)
- 25 % de 100 usuarios generando a la vez (S = 25) · L = 4.096 tokens · KV en FP16
- KV/token = 2 × 32 × 8 × 128 × 2 B = 128 KiB · KV/secuencia = 512 MiB

Escenario	Pesos	KV cache	Margen	Total	GPU que cabe
FP16 · 25 activas	~15 GiB	12,5 GiB	~4 GiB	~31 GiB	Una de 40–48 GB
INT4 · 25 activas	~4,5 GiB	12,5 GiB	~2,5 GiB	~20 GiB	Una de 24 GB
FP16 · 100 activas	~15 GiB	50 GiB	~10 GiB	~75 GiB	Una de 80 GB o dos

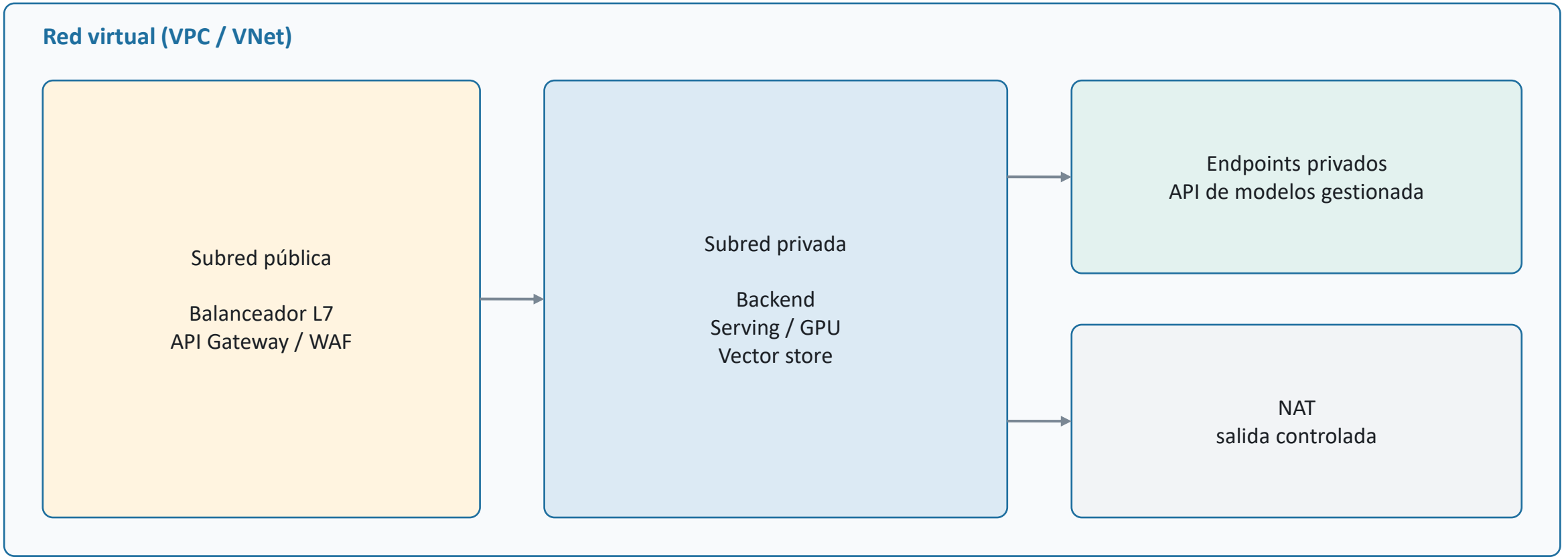
Para 99,9 %: dos réplicas en zonas distintas → el doble de GPU. Ejemplo académico, no medición.

Cuantización

Precisión	Bytes por parámetro	Modelo de 8 B	Efecto
FP16 / BF16	2	~15 GiB	Referencia de calidad
INT8 / FP8	1	~7,5 GiB	Menos memoria, decode más rápido
INT4	~0,5 + metadatos	~4,5 GiB	Permite CPU o GPU pequeñas; posible pérdida de calidad

Trade-off: la pérdida de calidad depende del método, del modelo y de la tarea → medir con evaluación propia.

Red: dónde vive cada componente



Ajustar timeouts de inactividad del balanceador para streaming.

Almacenamiento y vector stores

Tipo	Contenido	Consideración
Almacenamiento de objetos	Documentos fuente; pesos en self-hosting	Versionado, permisos; tamaño de pesos = arranque en frío
Volúmenes de bloque	Caché local de pesos en nodos GPU	Acelera arranque; atado a la zona
Vector store dedicado o búsqueda gestionada	Embeddings e índice semántico	Escala, búsqueda híbrida; otro servicio que operar
Extensión vectorial (pgvector)	Embeddings con metadatos y permisos	Simple si ya existe PostgreSQL; evaluar escala

Para el caso: priorizar el filtrado por permisos sobre el rendimiento bruto de búsqueda.

Docker, Kubernetes y serverless

Criterio	Contenedores serverless	Kubernetes gestionado	Funciones serverless
Carga operativa	Baja	Alta	Muy baja
Escalado a cero	Sí, con arranque en frío	Posible con componentes adicionales	Sí
GPU	En algunos servicios y regiones	Sí, con mayor control	No para modelos medianos
Duración de solicitudes	Limitada por el servicio	Sin límite propio	Limitada
Adecuado para	Backend; modelos pequeños	Plataforma de varios modelos	Ingesta por eventos

Docker es la unidad común: el mismo contenedor corre en el portátil, en serverless o en Kubernetes.

Alta disponibilidad y despliegue multizona

Componente	Estrategia	Costo relativo
Backend sin estado	Réplicas en varias zonas detrás del balanceador	Bajo
Serving propio con GPU	Al menos dos réplicas en zonas distintas	Alto: duplica GPU
API de modelos gestionada	Redundancia del proveedor; modelo o región de respaldo	Bajo en infraestructura
Vector store y estado	Servicio con réplicas multizona	Medio

99,9 % mensual ≈ 43 minutos de indisponibilidad: menos que una falla típica de zona.

Dos topologías de referencia para el caso

A · Gestionada

Gateway + backend en contenedores serverless multizona
API de modelos gestionada vía endpoint privado
Búsqueda vectorial gestionada con filtros por permisos
Identidad institucional + identidades gestionadas

Menor operación · costo variable · dependencia del proveedor

B · Self-hosted

Kubernetes con nodos GPU en dos zonas
Modelo open-weight con motor de serving propio
pgvector o vector store en la red privada
Observabilidad y autoscaling por cola

Más control · costo fijo · equipo de plataforma

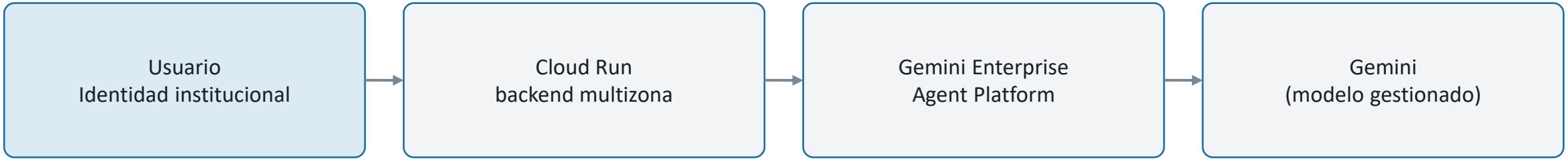
Unidad D · Comparación multicloud

Comparar por categoría de servicio, no por catálogo de productos. Ninguna nube es ganadora en abstracto.

Mapa de categorías de servicio (13-oct-2026)

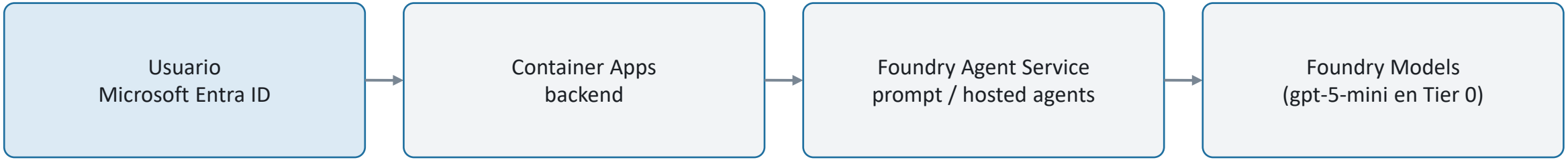
Categoría	AWS	Microsoft Azure	Google Cloud
Plataforma de modelos	Amazon Bedrock	Microsoft Foundry	Gemini Enterprise Agent Platform (antes Vertex AI)
Runtime gestionado de agentes	Bedrock AgentCore Runtime	Foundry Agent Service	Agent Engine / Agent Runtime
SDK de agentes del proveedor	Strands Agents	Microsoft Agent Framework 1.x	Agent Development Kit (ADK)
Contenedores serverless con GPU	ECS/EKS con instancias GPU	Azure Container Apps (GPU)	Cloud Run (GPU L4)
Kubernetes gestionado	Amazon EKS	AKS	GKE
RAG / vectores gestionados	Knowledge Bases + S3 Vectors / OpenSearch	Azure AI Search	Vector Search; AlloyDB/Cloud SQL + pgvector
Identidad de cargas	Roles de IAM	Identidades gestionadas (Entra ID)	Cuentas de servicio + Workload Identity
Guardrails gestionados	Bedrock Guardrails	Azure AI Content Safety	Model Armor
Observabilidad	CloudWatch + AgentCore Observability	Azure Monitor + App Insights	Cloud Logging / Monitoring / Trace

Google Cloud: arquitectura de referencia del caso



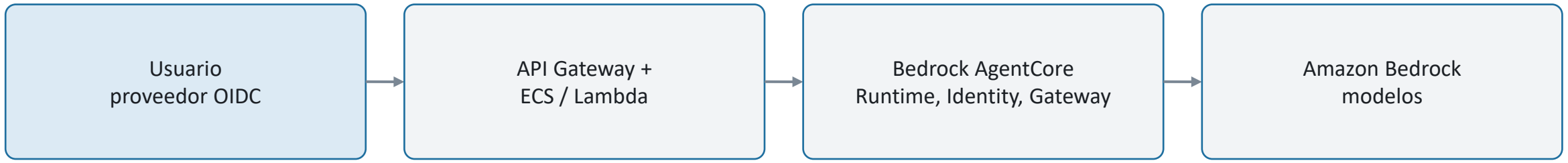
- Agente con ADK desplegado en Agent Engine / Agent Runtime (gestionado)
- Vector Search o AlloyDB/Cloud SQL con pgvector para RAG
- Secret Manager, cuentas de servicio con Workload Identity, Model Armor
- Trial: el crédito no cubre la Gemini API de AI Studio ni GPUs → usar la plataforma

Microsoft Azure: arquitectura de referencia del caso



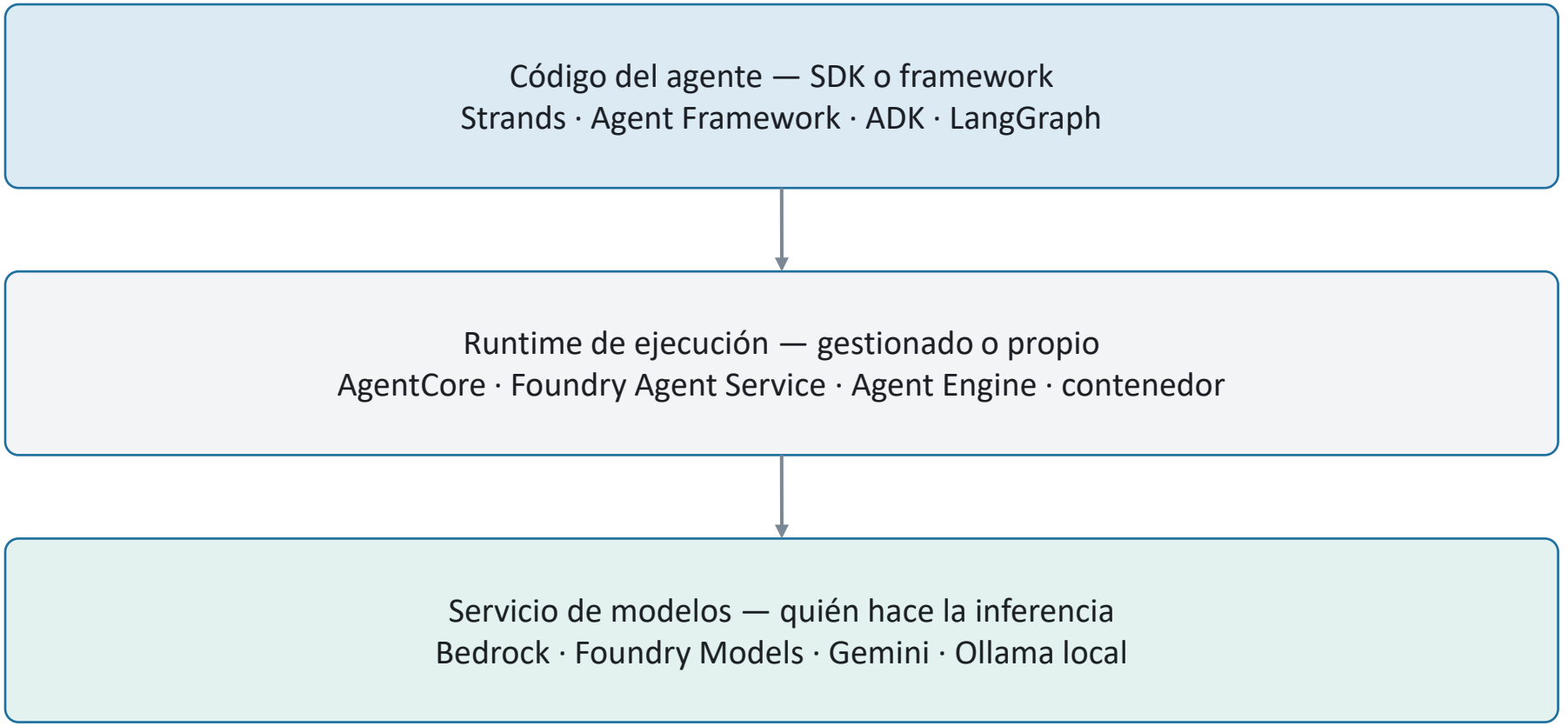
- Agent Framework 1.x para agentes en código; Semantic Kernel y AutoGen en mantenimiento
- LangGraph puede ejecutarse como hosted agent: framework externo, no oficial de Microsoft
- Azure AI Search para RAG · Key Vault · identidades gestionadas · API Management como AI gateway
- Trial: validar cuota (Free Tier / Tier 0) antes del laboratorio

AWS: arquitectura de referencia del caso



- Agente con Strands Agents desplegado en AgentCore Runtime (sesiones aisladas)
- Bedrock Knowledge Bases con S3 Vectors u OpenSearch para RAG
- Roles de IAM de mínimo privilegio · Secrets Manager · Bedrock Guardrails
- Cuentas nuevas: usar AgentCore; Bedrock Agents Classic ya no admite clientes nuevos

SDK, runtime y servicio de modelos: tres capas distintas



Responsabilidades y criterios de selección

Responsabilidad	API de modelos	Runtime de agentes	Self-hosting
Capacidad GPU y escalado	Proveedor	Proveedor	Organización
Sesiones y memoria del agente	Organización	Proveedor	Organización
Identidad, red, datos, prompts	Organización	Organización	Organización
Calidad y seguridad de las respuestas	Organización	Organización	Organización

- Criterios: ecosistema existente · modelos y región · residencia de datos · capacidades del equipo · costo total · portabilidad
- Estándares abiertos que reducen el lock-in: MCP (herramientas), A2A (agentes), OpenTelemetry (trazas)

Demo 3 · Agente gestionado en Microsoft Foundry

- Proyecto Foundry y despliegue de un modelo pequeño (gpt-5-mini)
- Prompt agent: instrucciones del asistente universitario
- Una herramienta de consulta (p. ej., calendario académico de ejemplo)
- Probar en el playground y revisar la traza de la ejecución
- Señalar los límites de confianza: identidad, herramienta, datos, modelo

Unidad E · Seguridad

En un LLM, los datos y las instrucciones viajan por el mismo canal.

Responsabilidad compartida en aplicaciones LLM

Siempre del proveedor	Siempre de la organización
Seguridad física e infraestructura	Identidades y permisos
En gestionados: modelo y disponibilidad	Clasificación y protección de datos
Parches de la plataforma gestionada	Configuración de red
	Prompts, documentos y herramientas del agente
	Evaluación de lo que el sistema responde

Identidad sin secretos estáticos y mínimo privilegio

Identidad	Mecanismo recomendado	Riesgo que evita
Usuario	SSO institucional con OIDC; RBAC por perfil	Acceso anónimo; información de otro perfil
Carga de trabajo	Identidad gestionada o workload identity	Claves de API en código o repositorios
Herramienta del agente	Credenciales de alcance mínimo; acceso en nombre del usuario	Agente con más permisos que el usuario

RAG: filtrar la recuperación por los permisos del usuario final, no por los del backend.

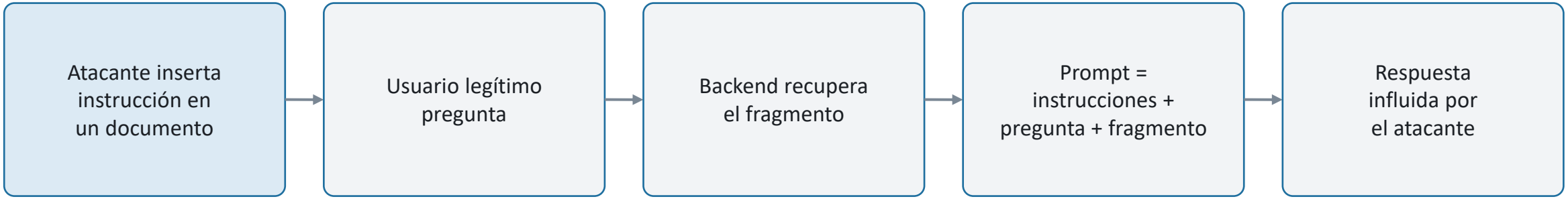
Red, cifrado y protección de datos

- Inferencia y datos en subredes privadas; endpoints privados hacia servicios gestionados
- TLS en todos los saltos; cifrado en reposo con llaves gestionadas por el cliente si la política lo exige
- Minimización: no enviar al modelo datos personales que la respuesta no necesite
- Residencia: algunos despliegues 'globales' pueden procesar fuera de la región del recurso
- Colombia: Ley Estatutaria 1581 de 2012 y decretos reglamentarios; análisis jurídico de la oficina competente

OWASP Top 10 para aplicaciones LLM 2025

Riesgo	En el asistente	Control principal
LLM01 Prompt Injection	Instrucciones en la pregunta o en un documento	Separar instrucciones y datos, guardrails, mínimo privilegio
LLM02 Sensitive Info Disclosure	Datos de otros estudiantes	Filtrado por permisos, minimización
LLM03 Supply Chain	Modelo o dependencia comprometidos	Fuentes verificadas, versiones fijadas
LLM04 Data/Model Poisoning	Documento alterado en el corpus	Procedencia y control de cambios
LLM05 Improper Output Handling	Salida ejecutada sin validar	Tratar la salida como no confiable
LLM06 Excessive Agency	Agente con permisos de escritura	Mínimo privilegio, aprobación humana
LLM07 System Prompt Leakage	Exposición de instrucciones	Sin secretos ni autorización en el prompt
LLM08 Vector/Embedding Weaknesses	Acceso indebido al índice	Control de acceso en el vector store
LLM09 Misinformation	Respuesta incorrecta de reglamento	RAG con citas, evaluación
LLM10 Unbounded Consumption	Abuso que agota cuota	Rate limiting, límites de tokens

Inyección indirecta a través de RAG



- No existe hoy un control que elimine por completo la inyección de prompts
- Objetivo de diseño: limitar el daño posible cuando ocurre
- Controles: procedencia de documentos, delimitación de datos, guardrails de salida, herramientas de mínimo privilegio

Defensa en profundidad, auditoría y gobierno

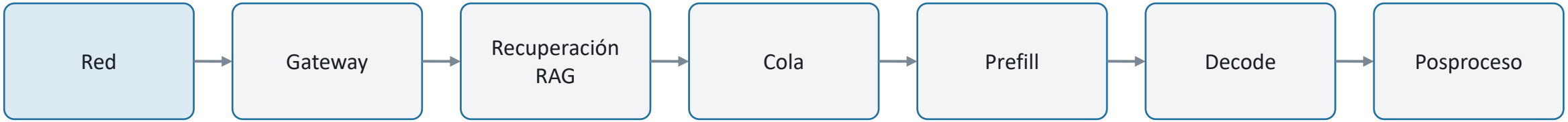
Capa	Controles
Perímetro	WAF, autenticación en el gateway, rate limiting
Entrada	Guardrails, límites de longitud, detección de inyección
Contexto	Filtrado por permisos, procedencia de documentos
Modelo y agente	Instrucciones de sistema, herramientas de mínimo privilegio, aprobación humana
Salida	Guardrails de salida, validación antes de ejecutar o mostrar
Operación	Logging seguro, trazas por solicitud, auditoría, respuesta a incidentes

Gobierno: NIST AI RMF y perfil de IA generativa · ISO/IEC 42001 (sistemas de gestión de IA)

Unidad F · Rendimiento, SLO y FinOps

¿Es suficientemente rápido? ¿Cuánto costará? Traducir supuestos a solicitudes y tokens.

Latencia end-to-end y métricas de inferencia



Métrica	Definición	Lo que percibe el usuario
TTFT	Tiempo hasta el primer token	Tarda en empezar a responder
TPOT	(Latencia total − TTFT) / (tokens de salida − 1)	Velocidad del texto en pantalla
Throughput	Tokens o solicitudes por segundo del sistema	No lo percibe; determina el costo

Reportar percentiles (p50, p95, p99), nunca solo promedios.

Concurrencia, colas y saturación

Ley de Little: $L = \lambda \times W$

Magnitud del caso	Cálculo	Resultado
Tasa de llegada λ	100 usuarios / 60 s	≈ 1,7 solicitudes/s
Solicitudes en curso L	1,7 × 10 s	≈ 17 (coherente con S = 25)
Tokens de entrada por minuto	100 × 3.000	300.000
Tokens de salida por minuto	100 × 300	30.000
Demanda en hora pico		≈ 330.000 tokens/min

El 90 % de los tokens son de entrada: recortar el contexto RAG ahorra más que acortar respuestas.

SLI, SLO y SLA

Concepto	Definición	Ejemplo para el asistente
SLI	Indicador medido	Proporción de solicitudes exitosas; TTFT p95
SLO	Objetivo interno	99,9 % mensual; TTFT p95 ≤ 2 s (propuesta a validar)
SLA	Acuerdo contractual con consecuencias	Lo que el proveedor garantiza a la universidad

Presupuesto de error: 0,1 % × 43.200 min ≈ 43 min al mes.

- Si el SLA del proveedor de modelos es menor que el SLO del asistente, el objetivo es inalcanzable sin redundancia

Costo por token frente a costo de infraestructura

	API gestionada	Self-hosting
Fórmula	Σ (tokens entrada × precio + tokens salida × precio)	réplicas × horas × precio GPU-hora + operación
Naturaleza	Variable, proporcional al uso	Fija por réplica y hora, se use o no
Caso (sin precios)	50.000 solicitudes/mes ≈ 165 M tokens	2 réplicas × 720 h = 1.440 GPU-horas

- FinOps: etiquetar recursos · presupuestos y alertas · límites de tokens por usuario
- Optimizar: recortar contexto RAG · enrutar preguntas simples a un modelo pequeño · caché de respuestas y de prompts

Demo 4 · Concurrencia frente a latencia

Concurrencia	TTFT p50 (ms)	TTFT p95 (ms)	E2E p95 (ms)	Solicitudes/s
1	113,9	132,0	1.110,8	0,91
2	115,3	118,7	1.095,7	1,83
4	1.193,9	1.212,8	2.187,8	1,83
8	1.757,5	3.394,2	4.372,0	1,83

Cifras sintéticas del simulador (capacidad 2): el throughput se estanca y el exceso se convierte en cola.

Taller · Topología base del asistente universitario

- Equipos de 3–4 personas · 14 minutos · plantilla D-28
- Entregable:
  - Diagrama con al menos 8 componentes, límites de red y de confianza
  - Decisión gestionado vs. autogestionado con 3 criterios (formato ADR)
  - Un SLO, una estimación de memoria o de tokens/min y 5 riesgos OWASP con su control
- Dos equipos exponen 3 minutos cada uno
- Rúbrica: componentes · justificación · seguridad · rendimiento y costo · comunicación

Síntesis: siete decisiones de una topología base

#	Decisión	Criterio principal
1	Comprar o producir inferencia	Volumen, control de datos, capacidad del equipo
2	Patrón de integración	Streaming para conversación; colas para tareas largas
3	Dimensionamiento	Pesos + KV cache + concurrencia + réplicas para HA
4	Topología de red	Componentes privados; endpoints privados
5	Identidad	OIDC para usuarios; sin secretos estáticos para cargas
6	Datos y RAG	Filtrado por permisos del usuario final
7	SLO y costos	Percentiles, presupuesto de error, tokens por minuto

Próxima sesión y laboratorios

- Repositorio: llm-infra-escalable (GitHub)
  - LOC-01 · Ollama + FastAPI + React + prueba de carga
  - GCP-A/B/C · Azure AZ-A a AZ-F · AWS-A a AWS-D · MC-01 comparativo
- Cuentas trial: crear la de cada nube la semana de su bloque; configurar presupuesto y limpiar recursos
- Tarea: completar el ADR del taller y la tabla de mapeo multicloud
- Sesión 2: continuidad sobre model serving, escalado y rendimiento

Referencias (1/2) · Documentación oficial y estándares

- Microsoft. Microsoft Agent Framework 1.0 (3-abr-2026). devblogs.microsoft.com/agent-framework
- Microsoft Learn. Foundry Agent Service overview; Azure OpenAI in Foundry Models quotas and limits (ago-2026)
- Google Cloud. Agent Development Kit overview; GPU support for Cloud Run services
- AWS. Amazon Bedrock AgentCore documentation; Bedrock Agents Classic maintenance mode; S3 Vectors GA (dic-2025)
- Strands Agents. Python Quickstart. strandsagents.com
- OWASP. Top 10 for LLM Applications 2025 (v2.0); Top 10 for Agentic Applications (dic-2025)
- NIST. AI RMF 1.0 (AI 100-1, 2023) y AI 600-1 (2024) · ISO/IEC 42001:2023 · Ley 1581 de 2012 (Colombia)

Referencias (2/2) · Literatura académica

- Vaswani et al. (2017). Attention Is All You Need. NeurIPS.
- Yu et al. (2022). Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI.
- Pope et al. (2023). Efficiently Scaling Transformer Inference. MLSys.
- Kwon et al. (2023). Efficient Memory Management for LLM Serving with PagedAttention. SOSP.
- Frantar et al. (2023). GPTQ. ICLR.
- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
- Yao et al. (2023). ReAct. ICLR · Greshake et al. (2023). Indirect Prompt Injection. AISec.